Data Science

Support Vector Machines

Sylvain BENOIT & Arthur THOMAS

Dauphine | PSL
UNIVERSITÉ PARIS

Support Vector Machines

A Support Vector Machine (SVM) is capable of performing non- and linear classification, regression, and even outlier detection. SVMs are particularly well suited for classification of complex but small- or medium-sized datasets.

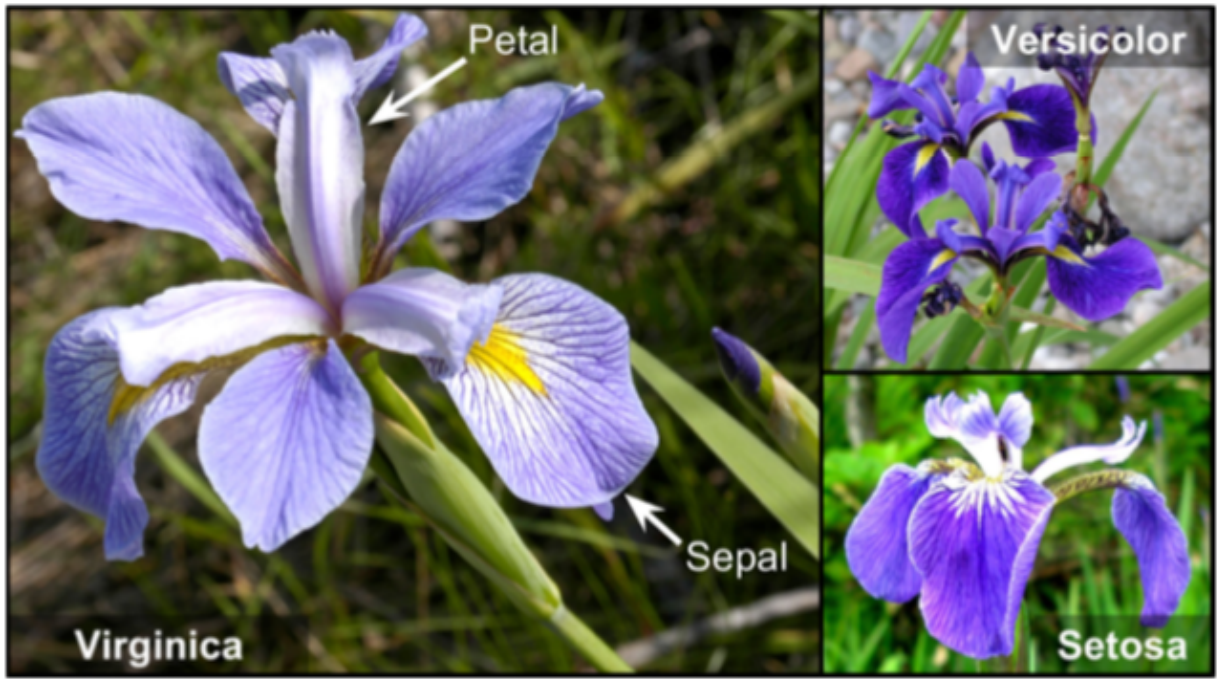
This section will explain the core concepts of SVMs.

Outline

1. SVM Classification
2. SVM Regression
3. Under the Hood

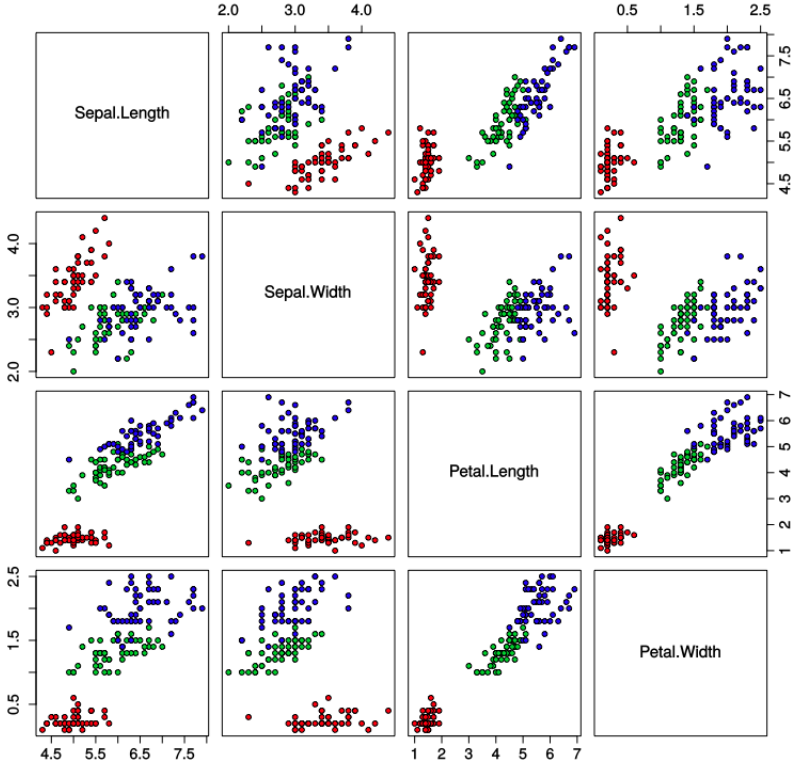
SVM Classification

The famous **iris dataset** contains the sepal and petal length and width of 150 iris flowers of three different species: Iris-Setosa, Iris-Versicolor, and Iris-Virginica.

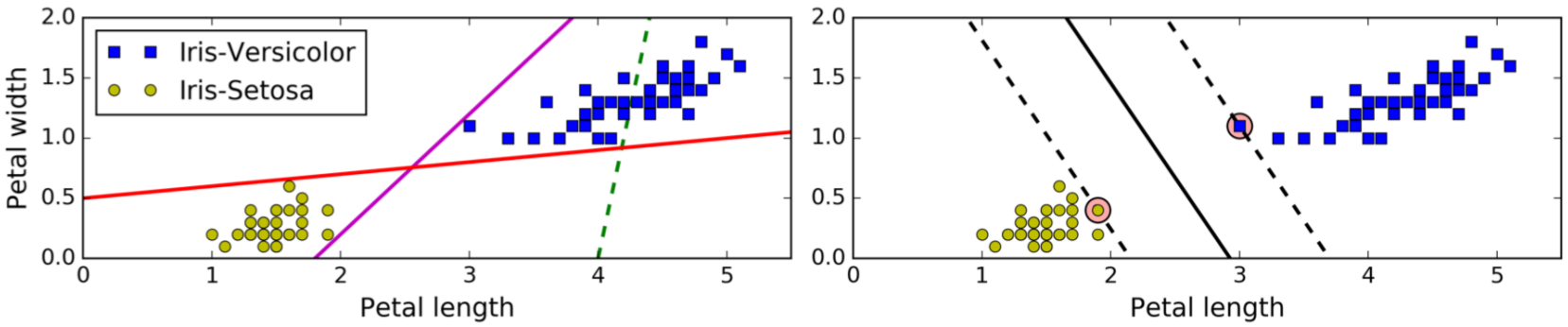


SVM Classification

The famous **iris dataset** contains the sepal and petal length and width of 150 iris flowers of three different species: Iris-Setosa, Iris-Versicolor, and Iris-Virginica.



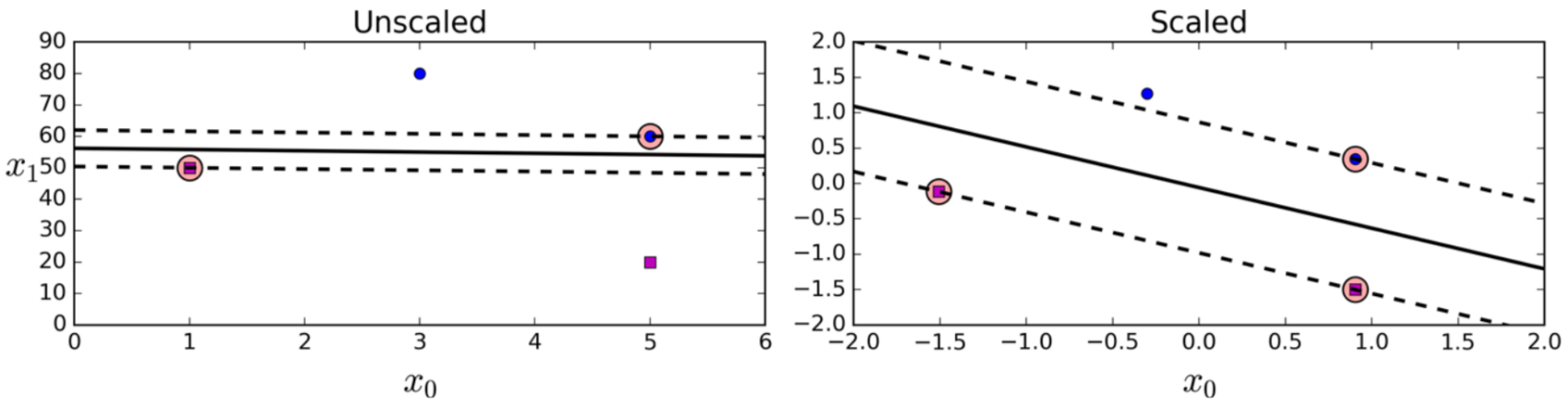
Basic Idea



- The two classes are *linearly separable*.
- SVM classifier fit the widest possible "street", represented by the parallel dashed lines on the right.
- The decision boundary is fully determined by the instances located on the edge of the street. These are called *support vectors*.

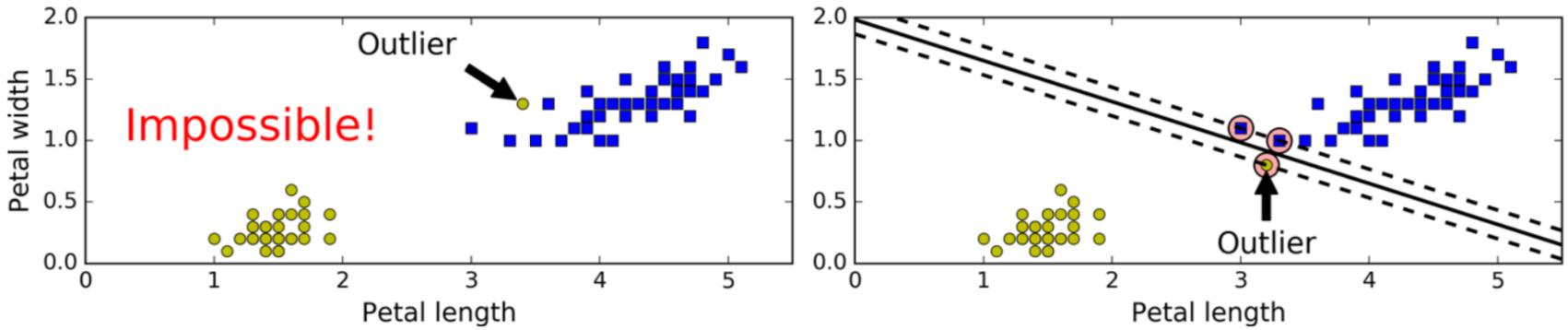
Sensitivity to Feature Scales

- Note that SVMs are sensitive to the feature scales.



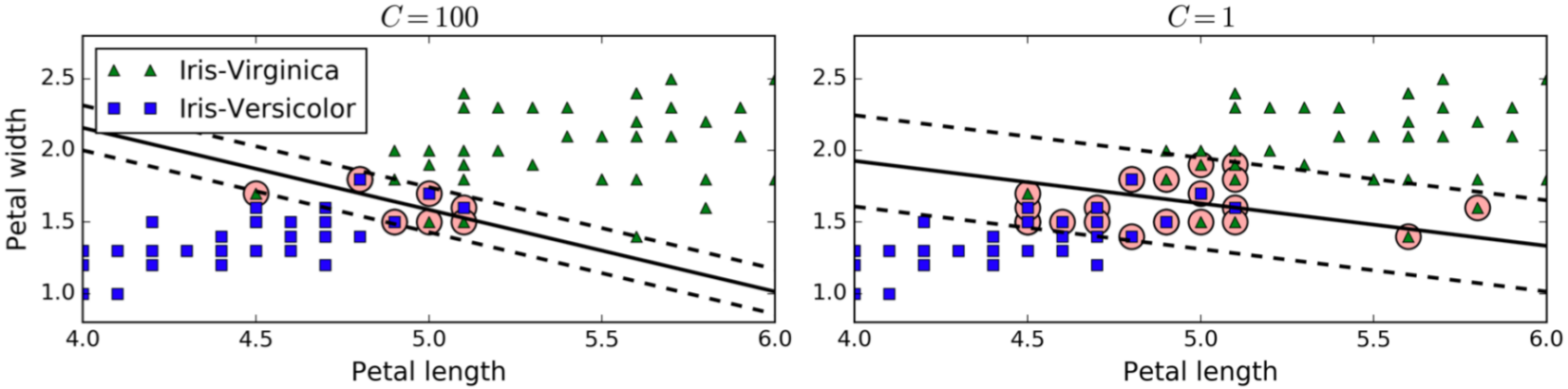
Hard Margin Classification

- If we strictly impose that all instances be off the street (and on the correct side), this is called *hard margin classification*.
  - Only works for linearly separable classes.
  - Is quite sensitive to outliers



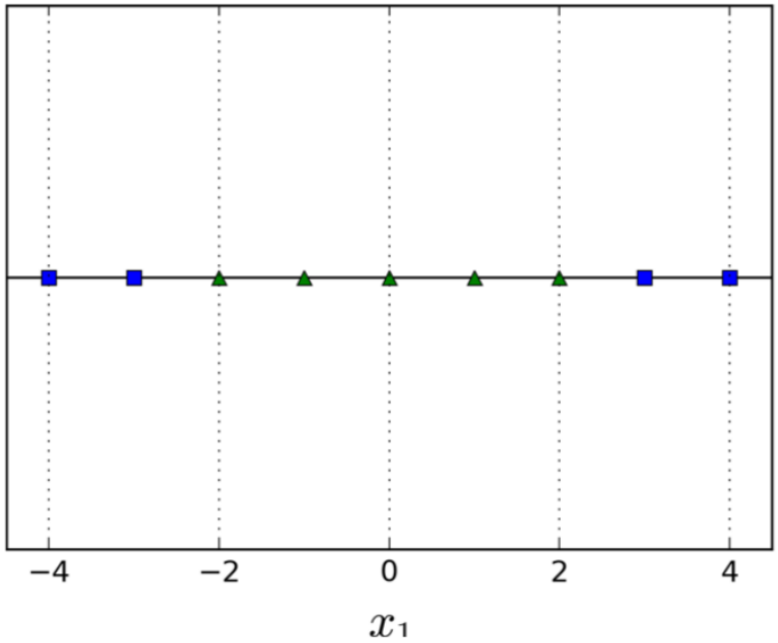
Soft Margin Classification

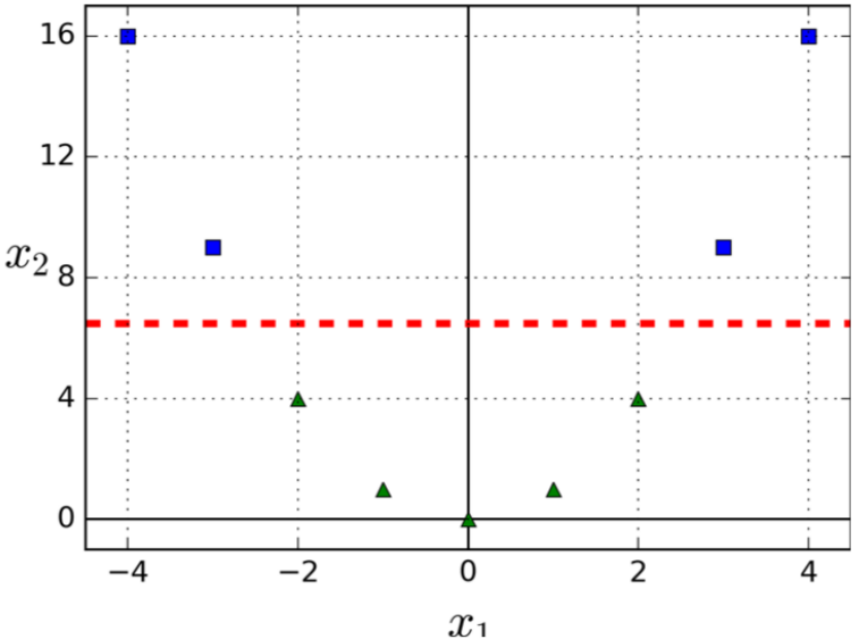
- A better objective is to find a good balance between
  - (a) keeping the street as large as possible and
  - (b) limiting the *margin violations*.
- This leads to *soft margin classification*.
- The hyperparameter controlling this balancing is denoted by C in the following.



Nonlinear SVM Classification

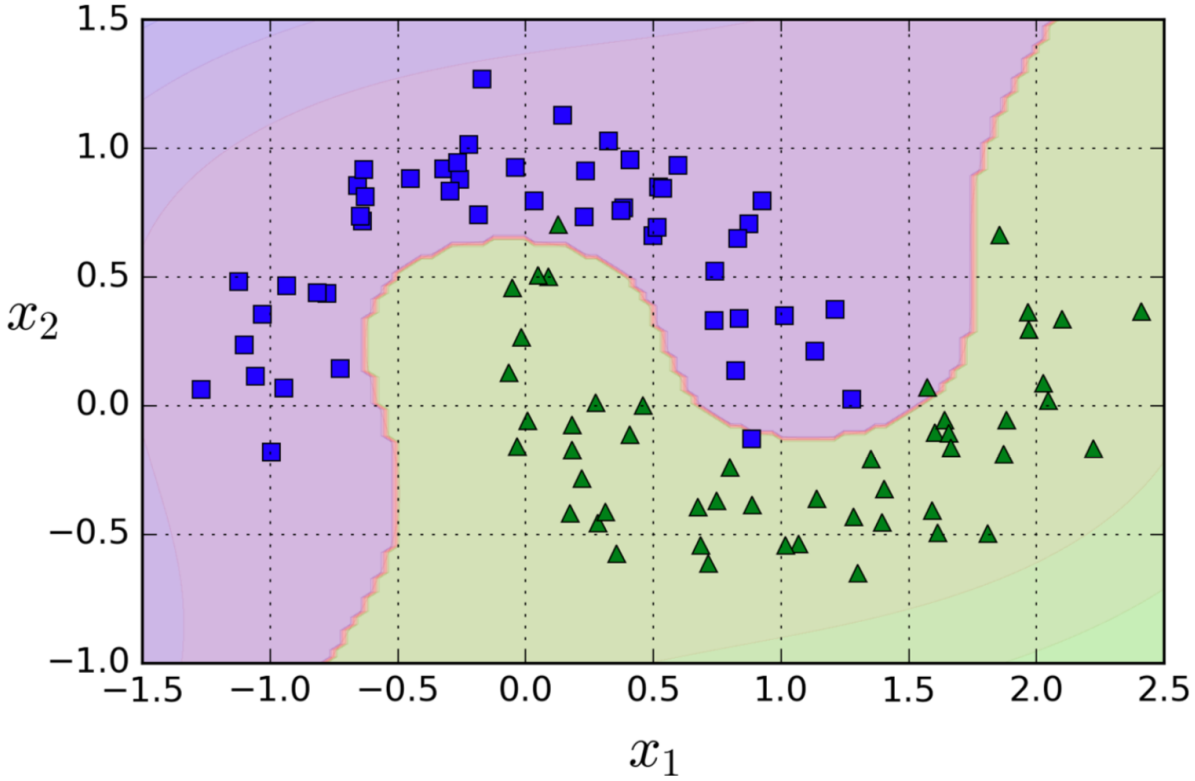
- One approach to handing datasets which are far from being linearly separable is to *add more features*.
- In some cases adding, for instance, polynomial features makes the data set linearly separable:





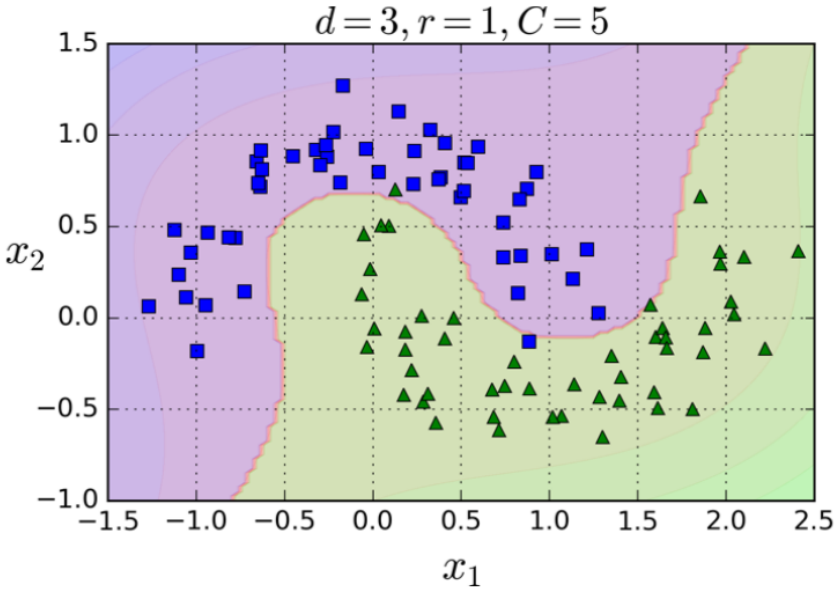
Polynomial Kernel

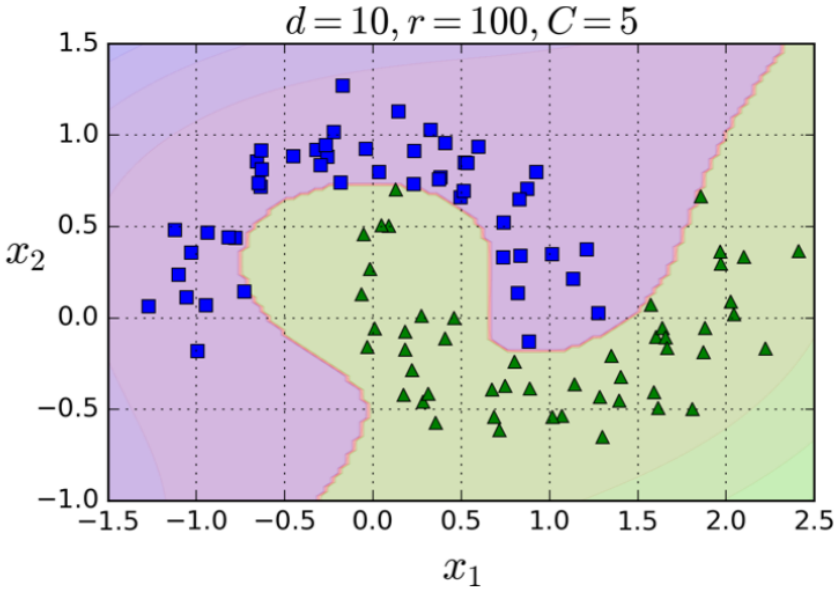
- Adding polynomial features is simple and can work great with all sorts of Machine Learning algorithms (not just SVMs).



Kernel Trick

- The *kernel trick* is a (miraculous) technique, which makes it possible to get the same result as if you added any polynomial features, even with very high degree polynomials, without actually having to add them.
- Below, d denotes the polynomial degree and r how much the model is influenced by high- vs low-degree polynomials.

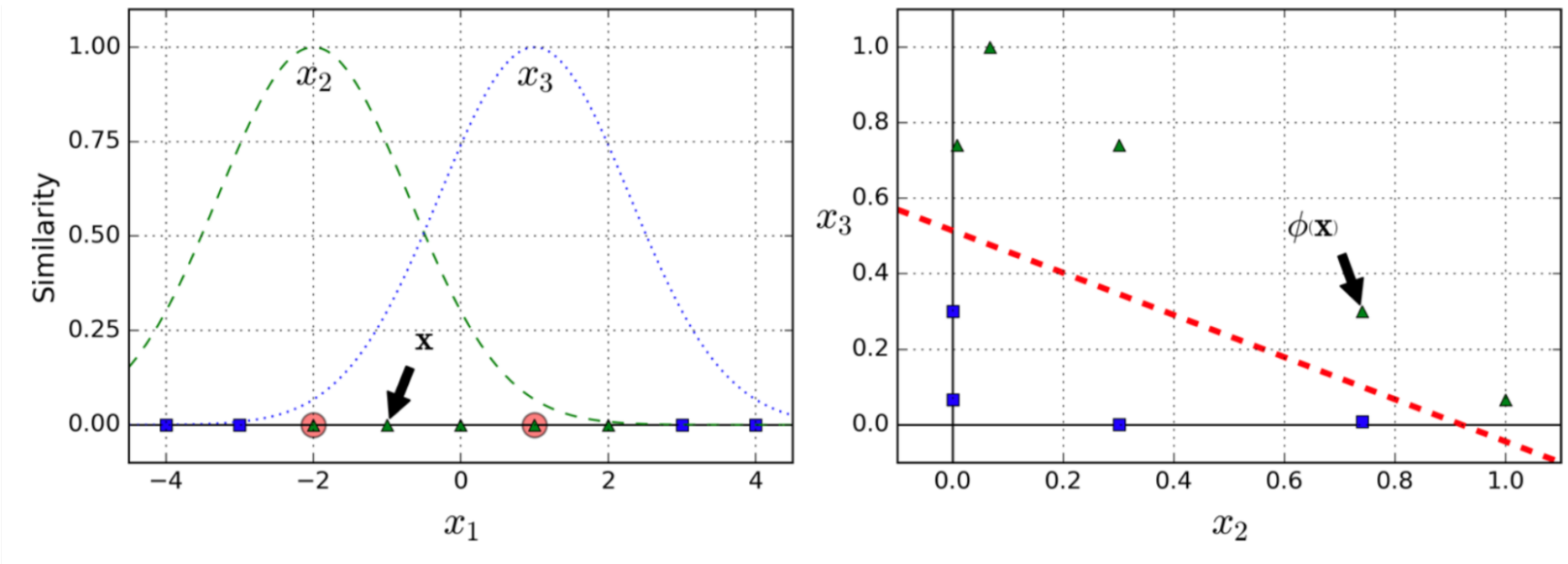




Adding Similarity Features

- Another technique is to add features that measure how much each instance resembles a particular *landmark*.
- For instance, we could add two landmarks at $\ell = -2$ and $\ell = 1$ and define the similarity function

$$\phi_\gamma(\mathbf{x}, \ell) = \exp\left(\gamma ||\mathbf{x} - \ell||^2\right).$$

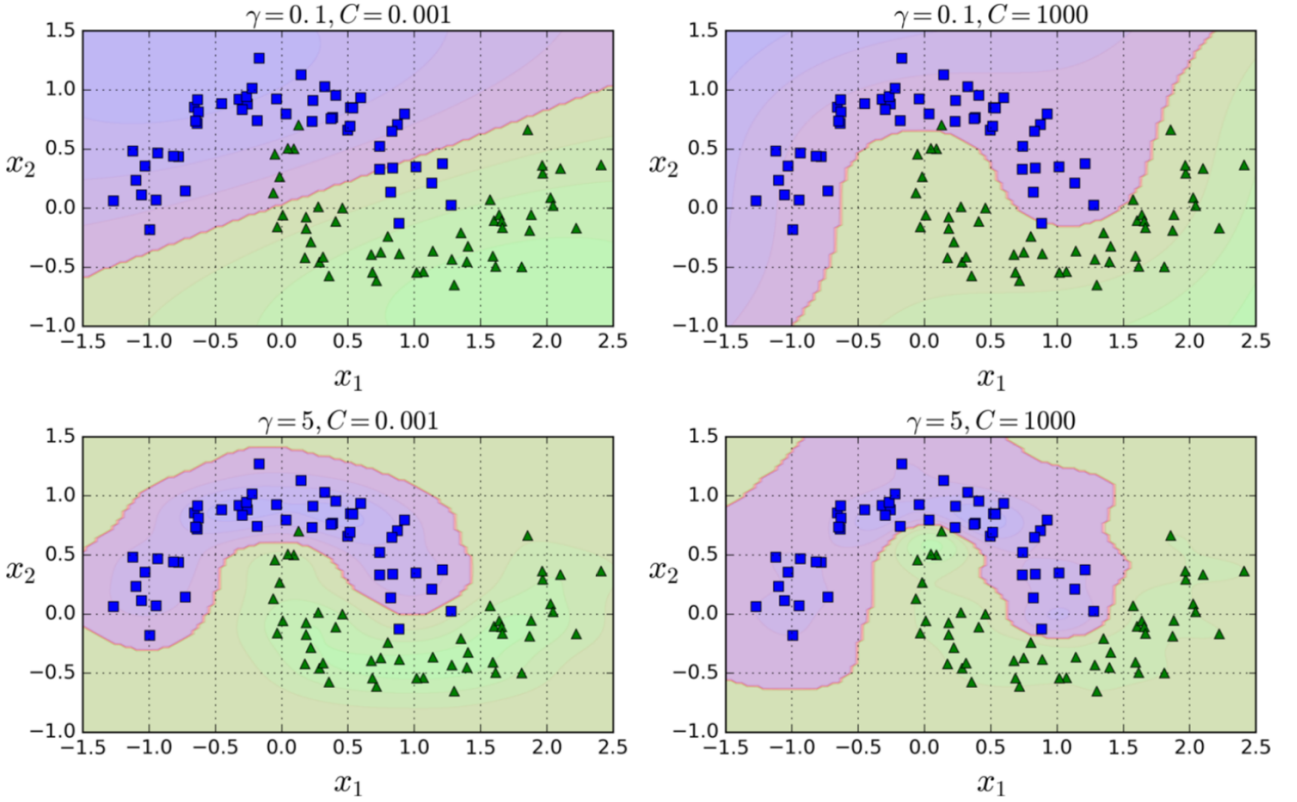


Adding Similarity Features

- How to select the landmarks?
  - Create a landmark at the location of each instance in the dataset.
  - Training sets with m instances and n features gets transformed to m instances and m features (assuming you drop the original features.
  - Increases dimensions, but most likely leads to linearly separable data.

Gaussian Radial Basis Function Kernel

- Again the *kernel trick* helps to not actually have to add all extremely many similarity features.

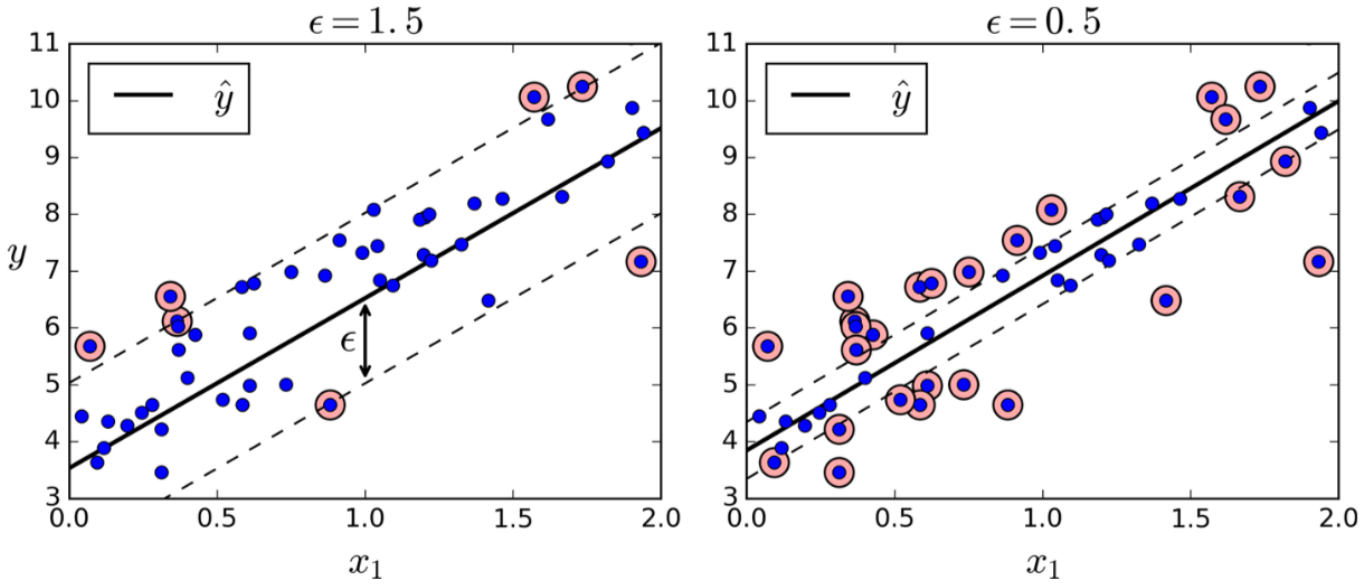


Outline

1. SVM Classification
2. SVM Regression
3. Under the Hood

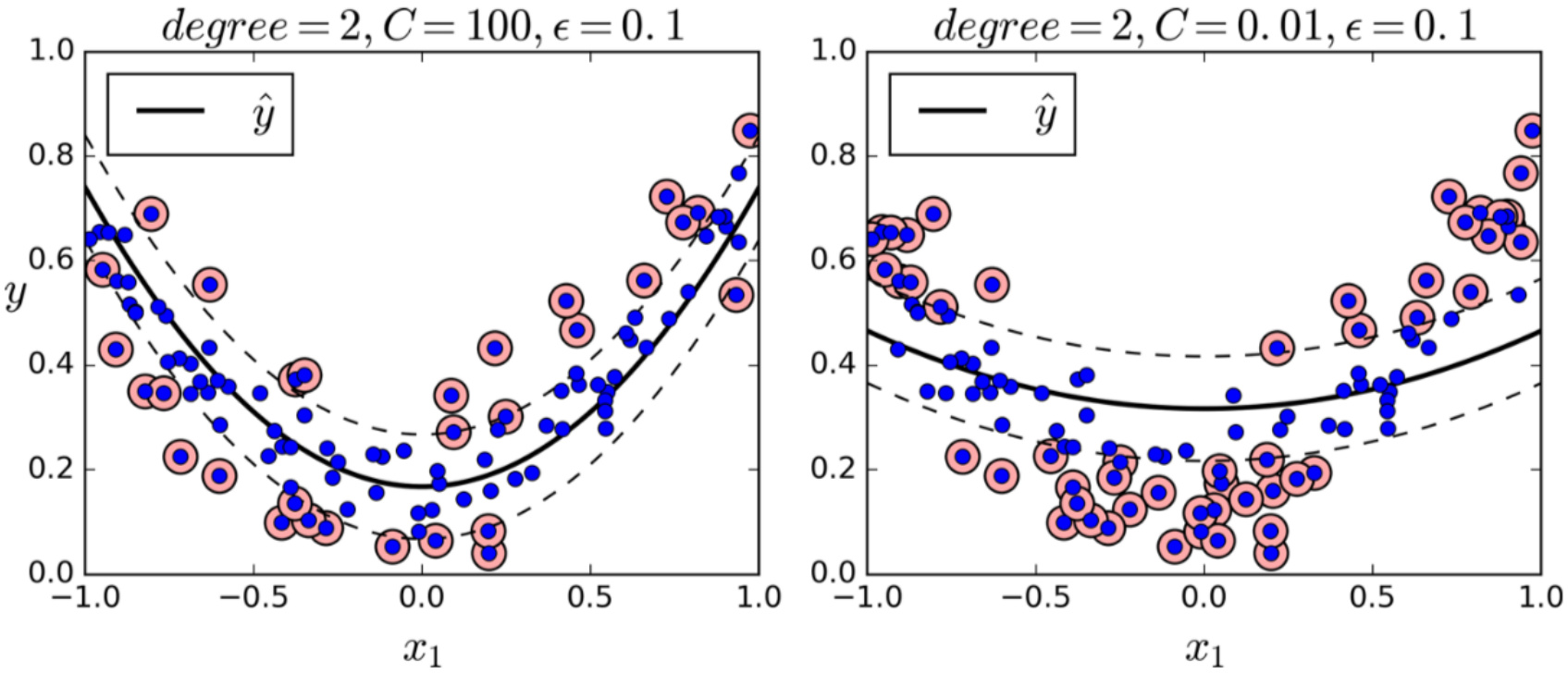
SVM Regression

- Instead of trying to fit the largest possible street between two classes, *SVM Regression* tries to fit as many instance as possible *on* the street while limiting marginal violations off the street.
- The hyperparameter ϵ controls the width of the street.



SVM Regression

- Similar techniques as presented above apply.



Outline

1. SVM Classification
2. SVM Regression
3. Under the Hood

Under the Hood

- Linear SVM classifier predicts the class $\hat{y}$ of a new instance $\mathbf{x}$ as follows

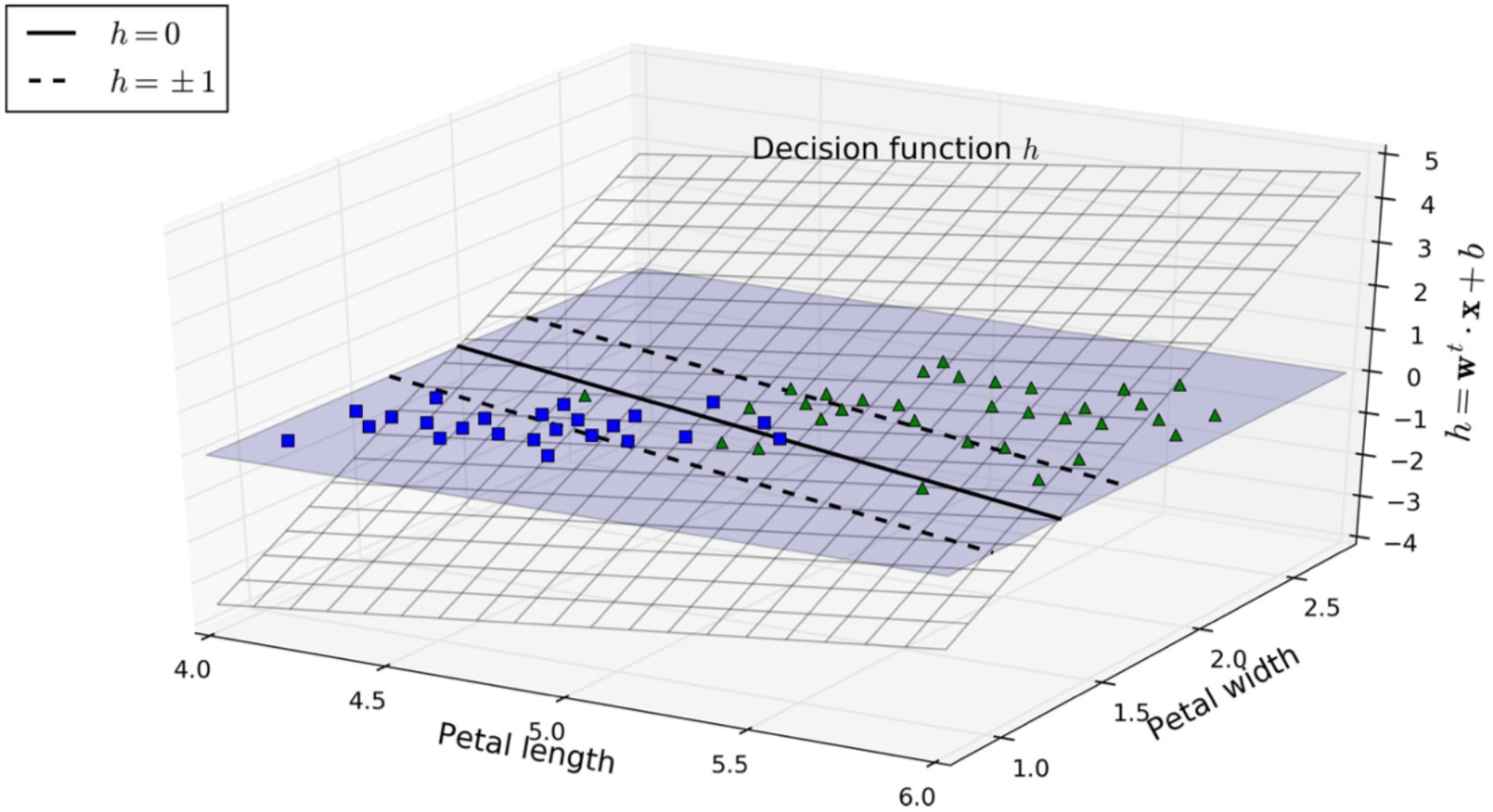
$$\hat{y} = \begin{cases} 0 & \text{if } \mathbf{w}^\top \mathbf{x} + b < 0 \\ 1 & \text{if } \mathbf{w}^\top \mathbf{x} + b \geq 0 \end{cases},$$

  where $\mathbf{w}$, the feature weight vector, and the bias b are the decision variables.
- The *decision function* is thus defined by

$$h = \mathbf{w}^\top \mathbf{x} + b.$$

- Training a linear SVM classifier means finding the value of $\mathbf{w}$ and b such that the separation implied by h is as discussed previously.

Decision Function of the Iris Dataset

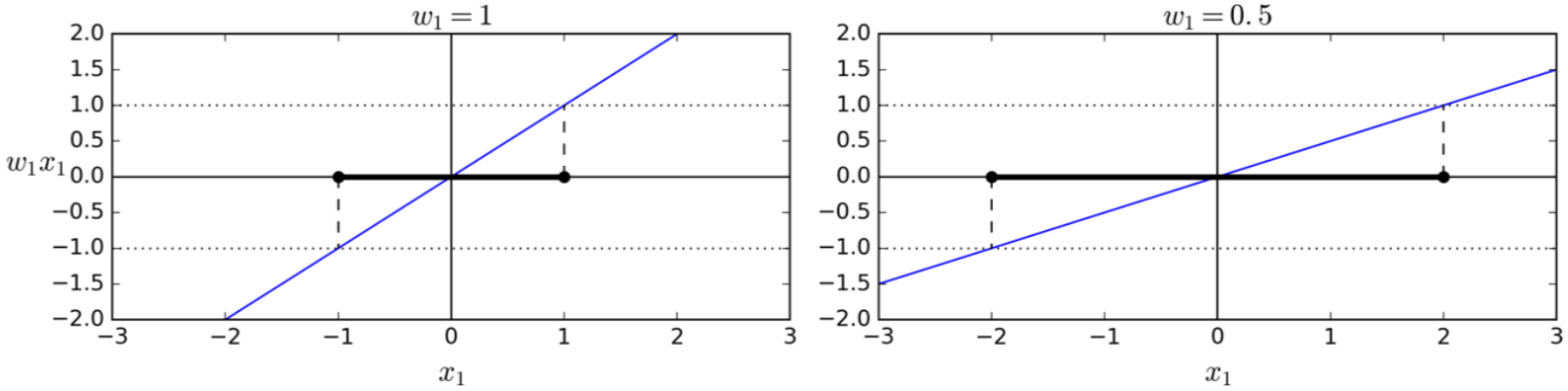


Training Objective

- We want to minimize

$$||\mathbf{w}||,$$

  since this gives the slope of the decision function h.
- A small $||\mathbf{w}||$ results in a larger margin. i.e. street!



Training Objective

- However, we want to avoid any margin violation. Hence, we define for each instance i

$$t^{(i)} = \begin{cases} -1 & \text{if } y^{(i)} = 0 \\ 1 & \text{if } y^{(i)} = 1 \end{cases}.$$

- Then,

$$t^{(i)}(\mathbf{w}^\top \mathbf{x}^{(i)} + b) \geq 1 \qquad \text{for all } i,$$

expresses this restriction.

Training Objective

- The hard margin linear SVM classifier objective is given by the quadratic program

$$\min_{\mathbf{w},b} \frac{1}{2}\mathbf{w}^\top\mathbf{w}$$
$$\text{s.t. } t^{(i)}(\mathbf{w}^\top\mathbf{x}^{(i)} + b) \geq 1 \qquad \text{for } i = 1, \ldots, m.$$

Training Objective

- The soft margin linear SVM classifier objective is given by the quadratic program

$$\min_{\mathbf{w},b,\zeta} \frac{1}{2}\mathbf{w}^\top\mathbf{w} + C\sum_{i=1}^{m}\zeta^{(i)}$$
$$\text{s.t. } t^{(i)}(\mathbf{w}^\top\mathbf{x}^{(i)} + b) \geq 1 - \zeta^{(i)} \qquad \text{for } i = 1,\dots,m,$$
$$\zeta^{(i)} \geq 0 \qquad \text{for } i = 1,\dots,m.$$

- The slack variable $\zeta^{(i)}$ measures how much the i-th instance is allowed to violate the margin

The Dual Objective

- The (strong) dual form of the hard margin linear SVM classifier objective is

$$\min_{\alpha} \frac{1}{2} \sum_{i=1}^{m} \sum_{j=1}^{m} \alpha^{(i)} \alpha^{(j)} t^{(i)} t^{(j)} (\mathbf{x}^{(i)})^{\top} \mathbf{x}^{(j)} - \sum_{i=1}^{m} \alpha^{(i)}$$
$$\text{s.t. } \alpha^{(i)} \geq 0 \qquad \text{for } i = 1, \ldots, m.$$

- The primal solution can be recovered from the dual as

$$\hat{\mathbf{w}} = \sum_{i} \hat{\alpha}^{(i)} t^{(i)} \mathbf{x}^{(i)},$$
$$\hat{b} = \frac{1}{n_s} \sum_{i, \hat{\alpha}^{(i)} > 0} \left(1 - t^{(i)} \left(\hat{\mathbf{w}}^{\top} \mathbf{x}^{(i)}\right)\right).$$

The Kernel Trick

- Assume you transform your data with a polynomial of degree 2. That is instead of $\mathbf{x}$ you consider

$$\phi(\mathbf{x}) = \phi\left(\begin{pmatrix} x_1 \\ x_2 \end{pmatrix}\right) = \begin{pmatrix} x_1^2 \\ \sqrt{2}x_1x_2 \\ x_2^2 \end{pmatrix}.$$

- In the dual formulation only $(\mathbf{x}^{(i)})^\top \mathbf{x}^{(j)}$ appears and this gets substituted by

$$(\phi(\mathbf{x}^{(i)}))^\top \phi(\mathbf{x}^{(j)}) = \cdots = \left((\mathbf{x}^{(i)})^\top \mathbf{x}^{(j)}\right)^2.$$

- Hence, you do not actually have to transform the data and add features!

The Kernel Trick

- In Machine Learning a *kernel* is a function $\phi(\cdot)$ capable of computing the dot product $\phi(\mathbf{a})^\top \phi(\mathbf{b})$ based only on the original vectors $\mathbf{a}$ and $\mathbf{b}$ without having to compute the transformation ϕ.
- Common kernels include
  - Linear: $K(\mathbf{a}, \mathbf{b}) = \mathbf{a}^\top \mathbf{b}$.
  - Polynomial: $K(\mathbf{a}, \mathbf{b}) = \left(\gamma \mathbf{a}^\top \mathbf{b} + r\right)^d$.
  - Gaussian RBF: $K(\mathbf{a}, \mathbf{b}) = \exp\left(-\gamma ||\mathbf{a} - \mathbf{b}||^2\right)$.
  - Sigmoid: $K(\mathbf{a}, \mathbf{b}) = \tanh\left(\gamma \mathbf{a}^\top \mathbf{b} + r\right)$.

Making predictions with a kernelized SVM

- One can show that

$$h_{\hat{\mathbf{w}},\hat{b}}(\phi(\mathbf{x}^{(n)})) = \sum_{i,\hat{\alpha}^{(i)}>0} \hat{\alpha}^{(i)} t^{(i)} K\left(\mathbf{x}^{(i)}, \mathbf{x}^{(n)}\right) + \hat{b},$$

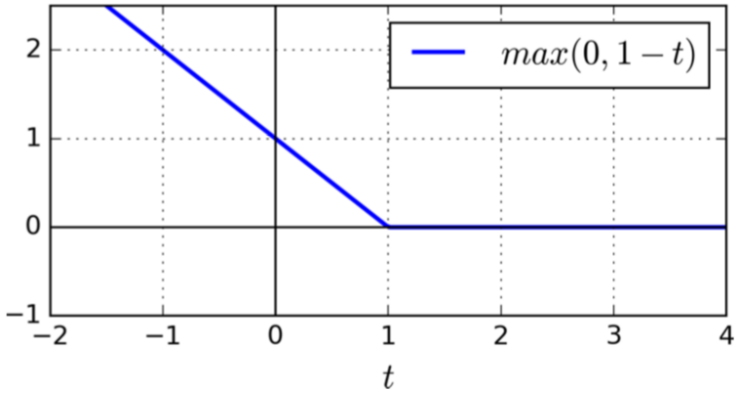
  where

$$\hat{b} = \frac{1}{n_s} \sum_{i,\hat{\alpha}^{(i)}>0} \left(1 - t^{(i)} \sum_{j,\hat{\alpha}^{(j)}>0} \hat{\alpha}^{(j)} t^{(j)} K\left(\mathbf{x}^{(i)}, \mathbf{x}^{(j)}\right)\right).$$

- Note that since $\hat{\alpha}^{(i)} \neq 0$ only for support vectors, making predictions involves computing the dot product of the new vector $\mathbf{x}^{(n)}$ with the support vectors only.

Further Details

- Online learning means learning incrementally as now instances arrive.
- For linear SVM classifiers, one method is to use Gradient Descent to minimize
- For linear SVM classifiers, one method is to use Gradient Descent to minimize

$$J(\mathbf{w}, b) = \frac{1}{2}\mathbf{w}^\top \mathbf{w} + C \sum_{i=1}^{m} \max\left(0, 1 - t^{(i)}(\mathbf{w}^\top \mathbf{x}^{(i)} + b)\right).$$

  - The convergence is much more slowly than the methods based on QP.
  - In machine learning, the *hinge loss* function $\max(0, 1-t)$ is often used in combination with Gradient Descent methods.